COL 1101

LECTURE 16

AUGUST 27, 2026

CONCURRENCY

LAST CLASS: RACE CONDITIONS IN CONCURRENT PROGRAMS

Thread A

line : balance += amt (100)

Thread B

balance += amt

Step	Thread A	Thread B	Balance
1.	read balance		100
2.		read balance	100
3	add amt		100
4		add amt	100
5	write balance		200
6		write balance	2

RACE CONDITIONS

Race condition : correctness depends on relative timing of operations performed by multiple threads

3 essential conditions

1. Shared data / memory
2. Mutable
3. Concurrent access

HOW TO HANDLE RACE CONDITIONS ?

The answer differs for the two programming paradigms.

Case 1 : Imperative / Object-oriented programming

↳ Tell what is happening → do this ; do this ; do this

HOW TO HANDLE RACE CONDITIONS ?

The answer differs for the two programming paradigms.

Case 1 : Imperative / Object-oriented programming

Recall the 3 conditions

1. Shared data / memory
2. Mutable
3. Concurrent access.

HOW TO HANDLE RACE CONDITIONS?

The answer differs for the two programming paradigms.

Case 1: Imperative / Object-oriented programming

Recall the 3 conditions

1. Shared data / memory
2. Mutable
3. Concurrent access. X

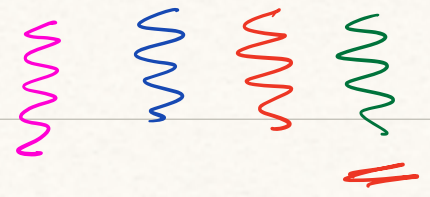
Add order among the threads through synchronization primitives

SYNCHRONIZATION PRIMITIVE EXAMPLE : LOCK

→ busy wait
→ yield

```
counter = 0
lock = threading.Lock()
with lock: acquire a lock
    counter += 1
```

threads wait for their turn





CRITICAL SECTION

counter += 1

LOCK : TWO IMPORTANT ASPECTS

```
lock = threading.Lock()

class Account:
    def __init__(self, balance):
        self.balance = balance
    def deposit (self, amt) :
        with lock:
            t = self.balance
            time.sleep(0)
            self.balance = t + amt
```

```
def work():
    for _ in range(10000):
        acc.deposit(10)
```

① What to lock ? — part of the code

↳ because it can have performance

def work
    with lock:
        for . . .
            acc.deposit(10)

LOCK : TWO IMPORTANT ASPECTS

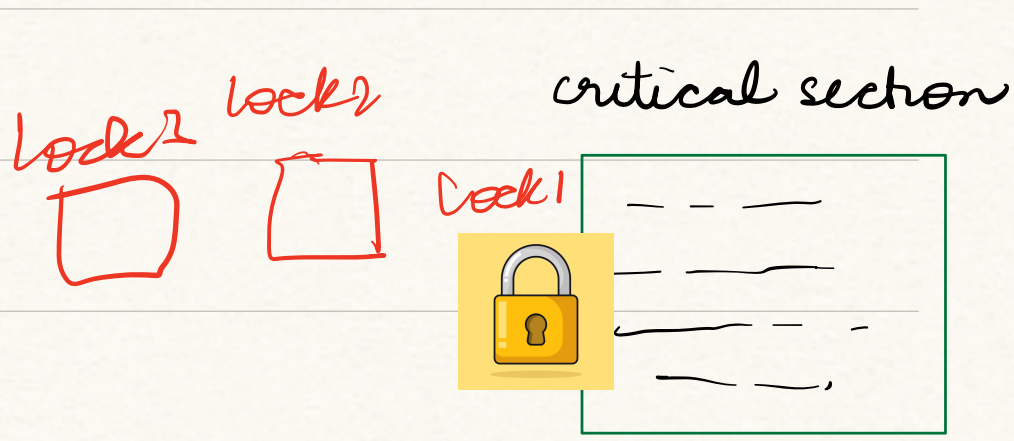
```
lock = threading.Lock()

class Account:
    def __init__(self, balance):
        self.balance = balance
    def deposit (self, amt) :
        with lock:
            t = self.balance
            time.sleep(0)
            self.balance = t + amt
```

```
def work():
    for _ in range(10000):
        acc.deposit(10)
```

① . What to lock ?

② . Locking creates contention

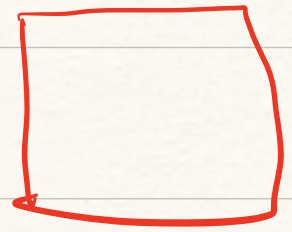


LOCK : TWO IMPORTANT ASPECTS

lock1 ← COND1

lock2 ← COND2

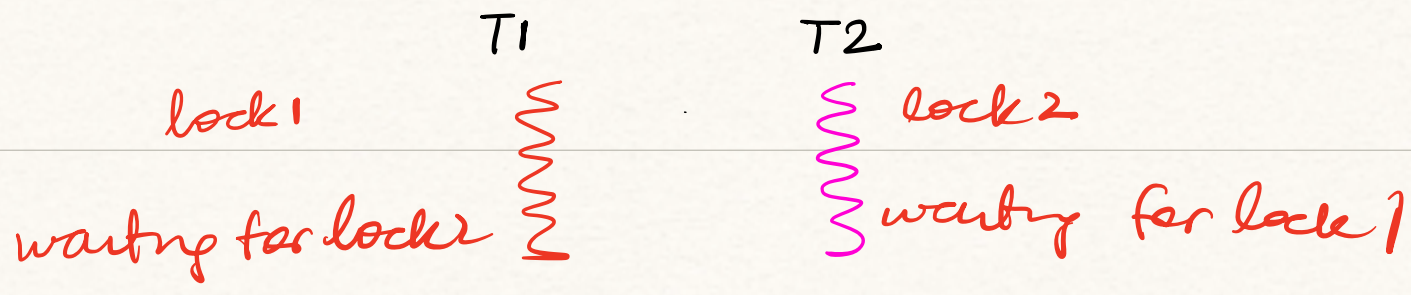
lockj ← COND3



1. What to lock ?

2. Locking creates contention

and sometimes deadlock



OTHER SYNCHRONIZATION PRIMITIVES

Variants of locks

1. Semaphores
2. Conditional wait
3. Mutex etc.

You will meet them in Operating Systems Course!

How to handle race conditions?

Case #1: Imperative programming / OOP

Use synchronization primitives.

Case #2: Functional programming

- must take data
- write pure functions

Recall 3 conditions:

1. Shared data
2. Mutable ✗
3. Concurrent access.

How to handle race conditions?

Case #1: Imperative programming / OOP

Use synchronization primitives.

Case #2: Functional programming

Recall 3 conditions:

1. Shared data
2. Mutable
3. Concurrent access.

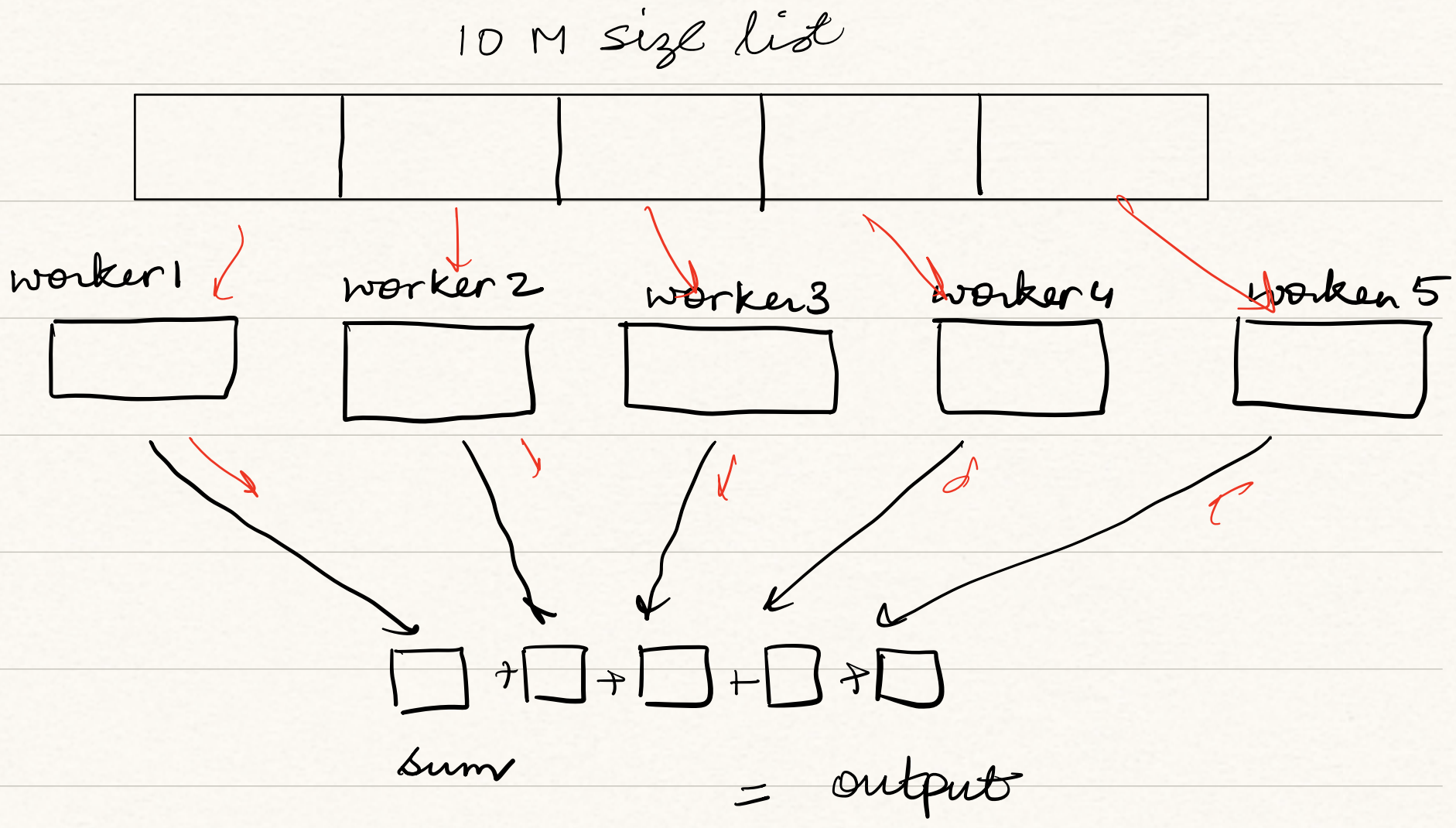
Immutable data is thread-safe by construction

↓

no race conditions!

Parallelization In Functional Programming

```python
if __name__ == '__main__':
    total = parallel_sum(list(range(10_000_001)))
    print(f"Parallel Result: {total}")
```



Parallelization In Functional Programming

```python
import multiprocessing as mp

def parallel_sum(data, cores=4):
    # Split data into manageable chunks
    size = max(1, len(data) // cores)
    chunks = [data[i:i + size] \
     for i in range(0, len(data), size)]

    # Map built-in 'sum' to each chunk in parallel,
    with mp.Pool(cores) as pool:
        return sum(pool.map(sum, chunks))

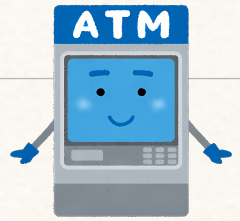
if __name__ == '__main__':
    total = parallel_sum(list(range(10_000_001)))
    print(f"Parallel Result: {total}")
```

Map Reduce

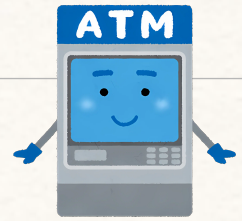
Hadoop

WHEN STATE MUST EVOLVE, PASS VALUES

same account, same instant!







fetch

balance -= 30

balance -= 30

same account

Account

Actor

market account slave

Let 1 actor maintain the state /entity

& everybody else communicate

the updates to that actor

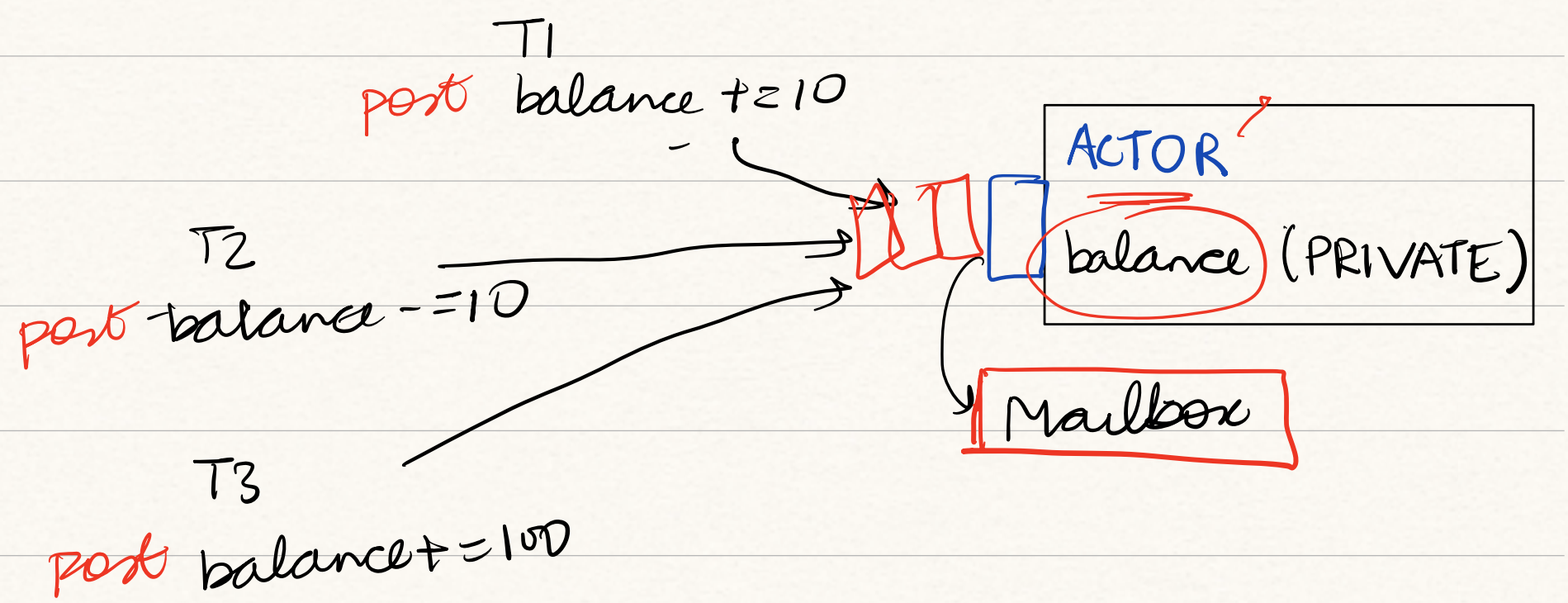
WHEN STATE MUST EVOLVE, PASS VALUES

Do not communicate by sharing memory,
share memory by communicating

WHEN STATE MUST EVOLVE, PASS VALUES

Do not communicate by sharing memory,
share memory by communicating

ACTOR MODEL



WHEN STATE MUST EVOLVE, PASS VALUES

Do not communicate by sharing memory,

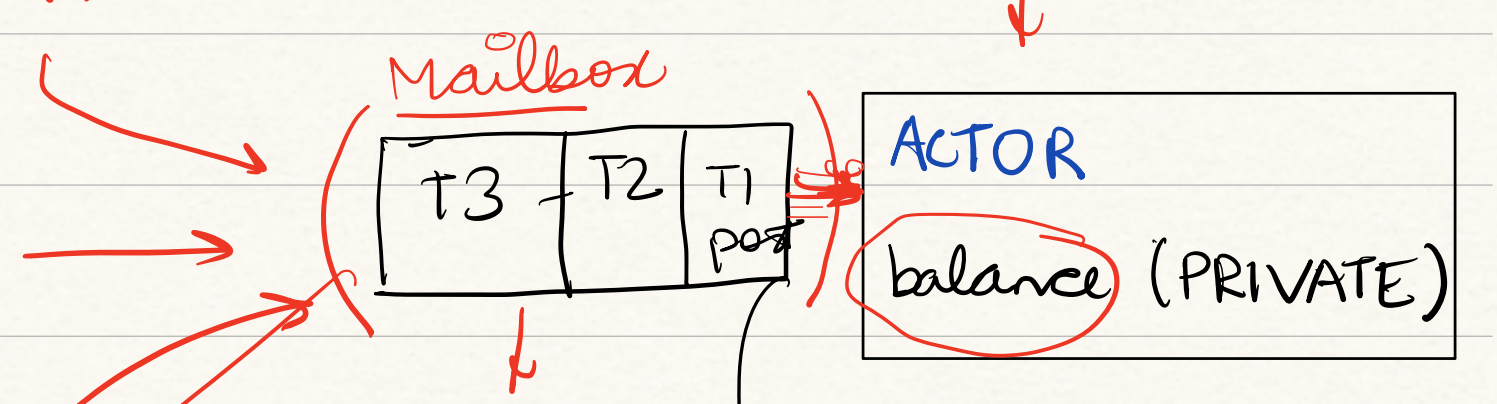
share memory by communicating

ACTOR MODEL

↳ both for updating/matching

↳ accessing the value

(Single-threaded)

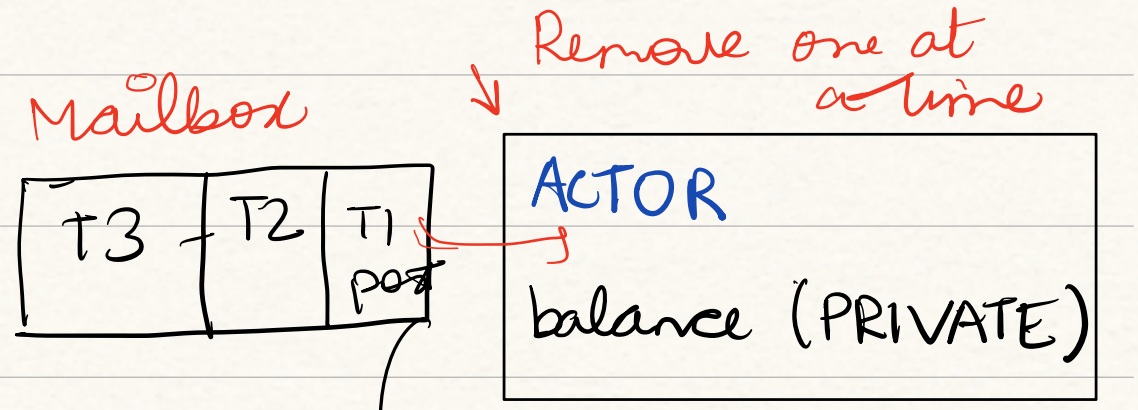


WHEN STATE MUST EVOLVE, PASS VALUES

Do not communicate by sharing memory,

share memory by communicating

ACTOR MODEL



Actor serializes

access to its state

but different from

locks